COSC 232 – Information System Security

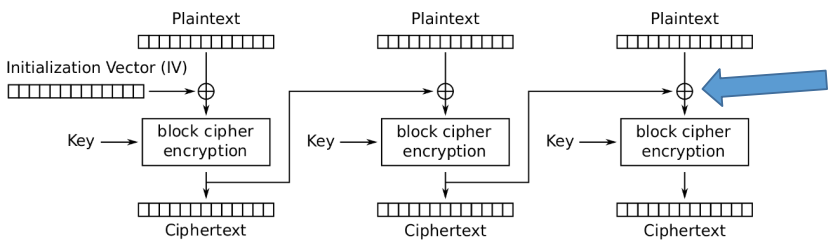
Winter 2026

Last Lecture Recap

- Last lecture, we introduced a different kind of cipher: the ***block cipher***
  - In a ***stream cipher***, we generate a ***key*** with the same length as our ***plaintext***
  - In a ***block cipher***, we cut the plaintext into ***blocks*** that match the length of the ***key***
- This allows for a much smaller key that can more readily be stored, so no need for ***PRNG key-stretching functions***, instead we have ***block permutations***

Last Lecture Recap: Modes of Operation

Common ***modes-of-operation*** include ***CBC***, ***ECB***, and ***Counter*** (***CTR***) modes

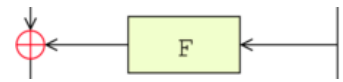


Cipher Block Chaining (CBC) mode encryption

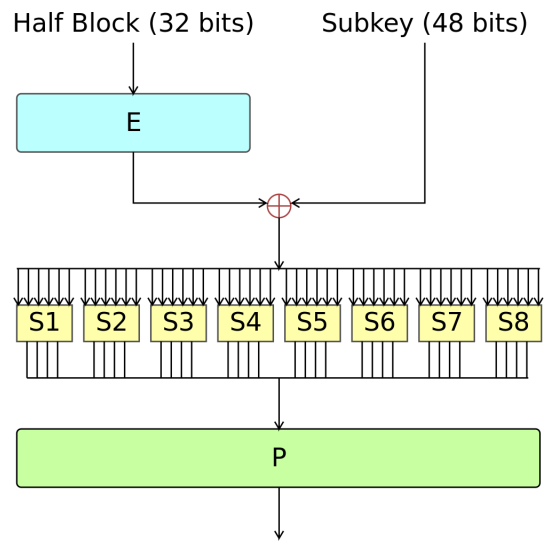
XOR (exclusive or) function used to combine previous ciphertext block with next plaintext block

- One major benefit of modern block ciphers is that they typically have many different ***modes of operation***
- One of the most common is ***cipher block chaining*** (CBC), which combines the encrypted ciphertext of the ***previous block*** into the ***plaintext*** of the next block
  - This improves security by modifying the plaintext before encryption – making it less likely for patterns in the plaintext to show up in the output

Last Lecture: The Block Function



- Last lecture, we also looked at the ***block encryption function*** inside of ***DES*** (i.e. the ***Feistel function***)
- This ***mixes*** bits of the ***half-block*** (32-bits) with bits of the ***subkey*** (48-bits), a form of ***permutation function***
- Most block ciphers use a ***block encryption function*** to mix the bits of the ***key*** and ***block*** together



The ***F*** block function in ***DES*** includes an ***Expansion*** (E), a ***XOR*** with the ***subkey***, and then a set of ***S-boxes*** that turn ***6-bit*** inputs into ***4-bit*** outputs

Another Example: The ChaCha20 Round

Initial state of ChaCha

"expa"	"nd 3"	"2-by"	"te k"
Key	Key	Key	Key
Key	Key	Key	Key
Pos.	Pos.	Nonce	Nonce

- Another interesting example of block cipher encryption function is that used in ***ChaCha20***
- It uses a ***state*** of ***16*** 32-bit ***words***, including a ***key***, ***counter (position)***, ***nonce***, and a fixed string (“expa” “nd 3” “2-by” “te k”)
- It then uses a function that takes and mixes ***4*** words at a time, aptly named the ***quarter-round function***
- The quarter-round uses a simple ***ARX*** function: ***Add, Rotate, and Xor*** – simple and ***fast***

Nothing Up My Sleeve Numbers

A random static default would be suspicious – why was it picked? Does it weaken the cipher?

- The first row of 32-bit values in the ***ChaCha/Salsa*** state is “expa” “nd 3” “2-by” “te k”, or “***expand 32-byte k***” – why is that there?
- This is a ***nothing up my sleeve*** number – i.e. a ***static default value*** that is set to a conspicuous value to allay suspicious that you might have introduced a ***backdoor***
- Other common values include ***Mersenne primes*** or the ***floor of sines*** – values that are derived from basic mathematics

A Full ChaCha Round

The ***ARX*** round function used by ChaCha20 for each round

```
a += b; d ^= a; d <<<= 16;
c += d; b ^= c; b <<<= 12;
a += b; d ^= a; d <<<= 8;
c += d; b ^= c; b <<<= 7;
```

- The quarter-round function can be seen top-right: the 4 inputs ***(a,b,c,d)*** are ***added*** to each other, ***XOR'd*** with each other, and ***rotated*** (via shifting)
- The quarter round is then (***confusingly!***) performed ***8 times***. Each ***even*** iteration mixes one of the ***4 columns of the state***, while each ***odd*** iteration mixes one of the 4 ***rows***
- This ***two-dimensional*** mixing technique ensures ***better mixing*** of the inputs

Two-Dimensional Mixing

This 2D mixing system ensures that all 16 words get a chance to mix properly with each other – always a challenge when developing a permutation function

Initial state of ChaCha

"expa"	"nd 3"	"2-by"	"te k"
Key	Key	Key	Key
Key	Key	Key	Key
Pos.	Pos.	Nonce	Nonce

```
a += b; d ^= a; d <<<= 16;
c += d; b ^= c; b <<<= 12;
a += b; d ^= a; d <<<= 8;
c += d; b ^= c; b <<<= 7;
```

The ***a, b, c, d*** of the ***ARX*** function depends on which iteration of the quarter-round you're on – it may be values from a ***row*** or a ***column***

- So, in the ***first*** of the iteration of the round function, the ***first column*** (expa + key + key + pos words) would be ***a, b, c, d*** and would be mixed using the ***ARX***
- In the ***second*** iteration would then mix the ***first row.*** Note that the first value in the first row (***expa***) is already mixed from the first iteration, so that helps mix the columns into the rows and vice-versa

Where's the Plaintext?

Initial state of ChaCha

"expa"	"nd 3"	"2-by"	"te k"
Key	Key	Key	Key
Key	Key	Key	Key
Pos.	Pos.	Nonce	Nonce

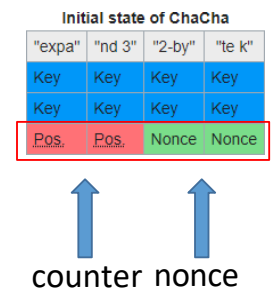
- You might notice though – the ***state*** of ***ChaCha20*** doesn't contain any ***plaintext***
- It only contains the ***nothing-up-my-sleeve*** value, the ***key***, the ***counter*** (***Pos***) and a ***nonce*** (random initialization vector)
- There's nowhere in the state for ***plaintext*** to fit in – so how does ChaCha's block encryption function work, if the block encryption function doesn't take a ***plaintext*** as input?

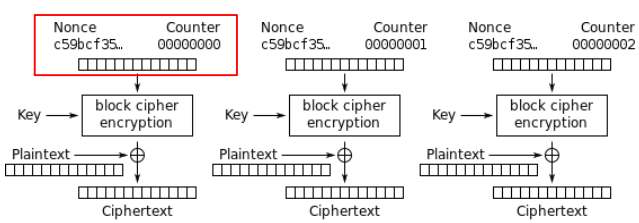
Salsa20 and the closely related **ChaCha** are stream ciphers developed by Daniel J. Bernstein.

Gotcha!

Stream ciphers live on, they just have to disguise themselves as block ciphers instead

- Remember when I mentioned ***block ciphers*** can be implemented as ***stream ciphers***?
- ChaCha20 (and Salsa20) are ***stream ciphers*** – the ***block encryption functions*** are used to generate a ***keystream***, which is then ***XOR***'d with the plaintext





Counter (CTR) mode encryption

- This is an example of a ***counter mode*** block cipher – it is performing encryption of ***an incrementing counter value*** to act as a ***PRNG***

Cryptographic Primitives

- At this point, you might be wondering if there are even rules anymore – a ***stream cipher*** based on a ***PRNG*** that is a ***block cipher*** in ***counter mode***?
- This is an important concept: we are dealing in ***cryptographic primitives*** – building blocks of ***complete encryption systems*** (Like DES or Salsa20)
- Modern encryption schemas can implement many different types of ***primitives*** all working in concert

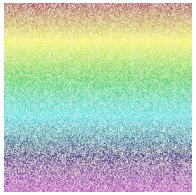
What Makes a Good Mixing Function?

- So, the ***round function*** the ***block encryption function*** that contains it are effectively ***mixing functions*** – they "stir" their inputs together – so what do we want in those functions?
  - ***First and foremost***, it should be difficult to determine the ***inputs*** based on the ***output*** – we need ***quality mixing*** (duh!)
  - A good mixing function should utilize the ***whole range*** of space it can use, equally – i.e., it shouldn't be biased towards certain output values (***equal distribution across output***)
  - It should also be ***sensitive*** – a single bit change in the ***input*** should propagate a ***larger*** change in the output (this is part of why we use multiple ***rounds – to ensure change propagation***)
    - As an example, a 1-bit change in ChaCha20 input impacts ~12 bits of state

Time and Time Again

Output of DJB2 hash function showing non-uniform output – D'oh!

- The qualities - having a ***uniform distribution output*** and ensuring ***change propagation*** - will appear in ***many areas of cryptography***; they can be hard to guarantee, but very desirable
  - Required for many cryptographic schemas in use – ***block and stream ciphers***, ***hash functions***, ***signatures and checksums***, and much more
- Again: a good mixing function is a ***cryptographic primitive*** that can be used in many different schemas



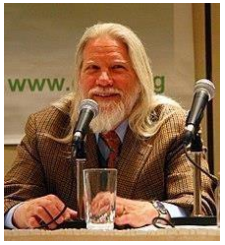
FNV-1a hash function – much better!

Stepping Back: DES and Block Cipher History

- The ***Data Encryption Standard,*** or ***DES***, was developed in the 1970's, out of the need for ***standardized algorithm*** for encrypting sensitive, but unclassified data (classified data encryption is a different ballgame here – this was more civilian)
- DES was developed based on earlier work by ***Horst Feistel***, a security researcher for IBM after whom the ***Feistel*** cipher is named
  - Feistel developed ***Lucifer*** in the late 60's and early 70's, one of the first civilian market block ciphers – this would be the basis of DES

The Standardization of DES

Santa Claus of cryptography Bailey Diffie



- DES was promptly certified in ***1976*** by the US government and published in the ***Federal Information Processing Standards*** (FIPS) for use on all unclassified data
- However, almost immediately it was challenged: ***Bailey Diffie*** and ***Martin Hellman*** argued it could be brute-forced, and even proposed a machine that could break it in a single day (albeit for millions of dollars)

Martin Hellman. Remember these guys, they'll be important later for developing ***public key cryptography***!

A Long, Healthy Life (for Cryptography)

- Despite concerns, ***DES*** lived a long, healthy life by most cryptography standards – it was ***reaffirmed multiple times***, last of which was in ***1999*** – more than ***two decades after it was introduced!***
- Even after ***Eli Biham*** and ***Adi Shamir*** brought differential cryptographic analysis (from the Enigma-breaking days) back into the spotlight in the 1990s, differential attacks on DES were still considered ***impractical***

What Changed?

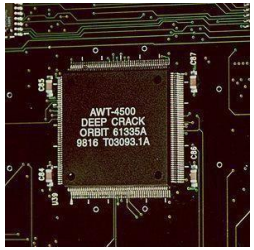
There were also legal limitations on ***key length*** at the time – cryptography that used longer keys was considered an ***armament*** and was subject to ***export controls*** as if it was an artillery piece or surface-to-air missile

- So, what brought about DES's downfall? It's key – only ***56 bits long***, very short by modern standards
- Computing power became cheap enough that dedicated chips could be built to brute-force DES encryption in a short period of time
- This culminated in a competition that saw DES cracked in ***an unprecedent amount of time, for relatively low cost***

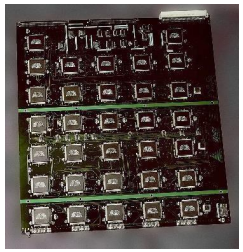
Record set in cracking 56-bit crypto

A joint effort between the Electronic Frontier Foundation and Distibuted.Net cracks the DES algorithm in less than 23 hours.

The Competition



- ***RSA***, an encryption company, wished to show that DES was insecure – so they offered ***$10,000*** prizes for breaking encrypted DES ciphertexts



- The first in 1997 was solved in ***96 days***, the next in 1998 was solved by ***distributed computing*** in ***39 days***, and the ***Electronic Frontier Foundation*** beat the third challenge in 1998 in just ***56 hours***
- In 1999, the EFF and the distributed computing team joined forces and cracked it in just ***22 hours***

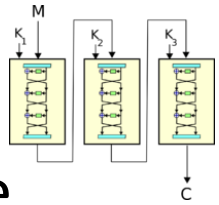
What Happened Next

TECHNOLOGY

TECHNOLOGY; U.S. Selects a New Encryption Technique

NY Times article from 2000 heralding AES

- Now, you might notice that I mentioned DES was ***reaffirmed in 1999*** – ***after*** it was known it could be brute-forced!
  - It got re-affirmed as ***Triple DES***, which performs DES on each block ***three times***, potentially using ***three different keys*** (56 * 3 = 168 bit key effectively)
- Triple DES is still "secure" by most standards, but is too slow – which resulted in the development of the ***Advanced Encryption Standard***, or AES!



Next Lecture

- Next lecture, we'll talk about ***AES*** – it's still considered ***the*** standard for symmetric block cryptography, and is still considered ***unbroken***
  - There are theoretical attacks, but none capable of breaking all ***14 rounds*** of mixing used in a ***256*** bit AES key
- We'll look at the ***internals*** of ***AES*** – what it's ***per-round*** mixing function looks like, and how it compares to the block functions we've already seen

Any Questions?